

Lecture Notes on Agentic AI

Jawwad Ahmed Shamsi
FAST-NUCES, Pakistan

August 28, 2026

5 Efficient Architectures and Model Reliability

This lecture closes an open loop from earlier in the course – the puzzle of how a model with trillions of parameters can run on a laptop – and opens a new, critical topic: why models sometimes confidently say things that are simply false. Learning objectives:

1. Explain quantization, why weights get rounded, and the memory savings it provides
2. Explain Mixture-of-Experts (MoE) as an alternative to dense architectures – specifically, which part of a Transformer layer it replaces, and how a routing decision is computed step by step
3. Combine quantization and MoE to fully resolve the Kimi K3 puzzle
4. Define hallucination, distinguish intrinsic from extrinsic hallucination, and explain why it happens
5. Explain why hallucination carries higher stakes for agents than for chatbots
6. Describe practical strategies for detecting and mitigating hallucination, in general and for agents specifically

5.1 Quantization

Definition

Quantization reduces the numeric precision used to store a model's weights – for example, storing each weight using 4 bits instead of 32 – drastically reducing memory use and inference cost, at the price of some – usually small – loss in quality.

5.1.1 What Does “Precision” Actually Mean?

A weight is just a number, like 0.7328. How that number gets stored determines how *exactly* it can be represented.

Example

Fewer bits means fewer possible values. With 4 bits, there are only $2^4 = 16$ distinct values available at all. If those 16 values are spread evenly across the range $[-1, 1]$, adjacent representable values are $2/15 \approx 0.133$ apart. A true weight of 0.7328 does not fall exactly on one of those 16 values – it gets **rounded** to the nearest one, in this case 0.733 (an error of only about 0.0002).

With 32 bits, there are 2^{32} – over 4 billion – distinct values available in the same range, so weights are stored with far finer precision and almost no rounding error. The tradeoff is exactly this: INT4 rounds every single weight slightly; FP32 barely rounds at all, but costs 8 times more memory to store.

Question

If every weight gets rounded, why doesn't the model just break? Individually, each rounding error is tiny – as the example above shows. A model has billions of weights, and these small, roughly random errors tend to partially cancel out across a full computation rather than compounding catastrophically. This is why quantization usually costs only a small amount of output quality, not a proportional collapse in it.

5.1.2 Worked Example: Memory Savings for a 7-Billion-Parameter Model

Example

- FP32 (32-bit, 4 bytes/parameter): $7\text{B} \times 4 \text{ bytes} = 28 \text{ GB}$
- FP16 (16-bit, 2 bytes/parameter): $7\text{B} \times 2 \text{ bytes} = 14 \text{ GB}$
- INT8 (8-bit, 1 byte/parameter): $7\text{B} \times 1 \text{ byte} = 7 \text{ GB}$
- INT4 (4-bit, 0.5 bytes/parameter): $7\text{B} \times 0.5 \text{ bytes} = 3.5 \text{ GB}$

Going from FP32 to INT4 shrinks memory usage by $8\times$, using exactly the rounding tradeoff described above – fewer representable values per weight, in exchange for a fraction of the memory footprint.

Question

Where does this fit into the Kimi K3 puzzle from earlier in the course? Quantization alone is only *half* the explanation for how a 2.8-trillion-parameter model can run in 8 GB of RAM. The rest of this lecture supplies the other half.

5.2 Dense vs. Mixture-of-Experts (MoE)

Definition

In a **dense** architecture, every parameter in the network participates in processing every token. In a **Mixture-of-Experts (MoE)** architecture, the network is split into many smaller sub-networks (“experts”), and a **router** selects only a small subset of experts to process each token.

5.2.1 What Exactly Gets Replaced?

This is the single most important clarification in this lecture, and the most commonly missed one: **MoE does not replace the whole Transformer block**. Recall the block-diagram from earlier in the course – each layer has an **attention** step and a **feedforward network (FFN)** step. MoE only touches the second one.

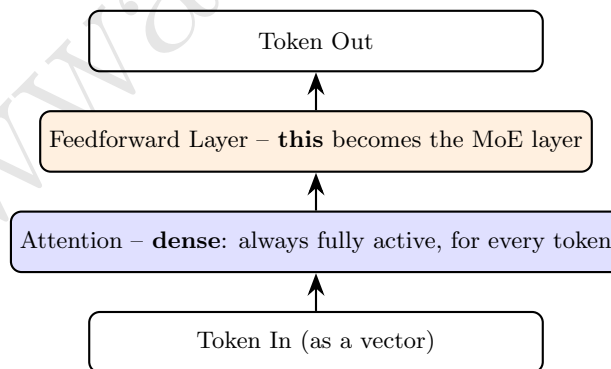


Figure 1: One Transformer layer. Attention always runs in full, for every token – it is never sparse. Only the feedforward step is replaced by a Mixture-of-Experts layer. The figure below zooms into that one orange box.

Question

So does the router decide which words to ‘pay attention’ to as well? No – that is a common mix-up. Attention (which token relates to which) is completely separate from MoE routing (which expert processes a token), and attention is never sparse in a standard MoE model. Only the feedforward sub-layer – the part that transforms each token’s vector independently – gets split into experts.

Now we zoom into that orange box specifically.

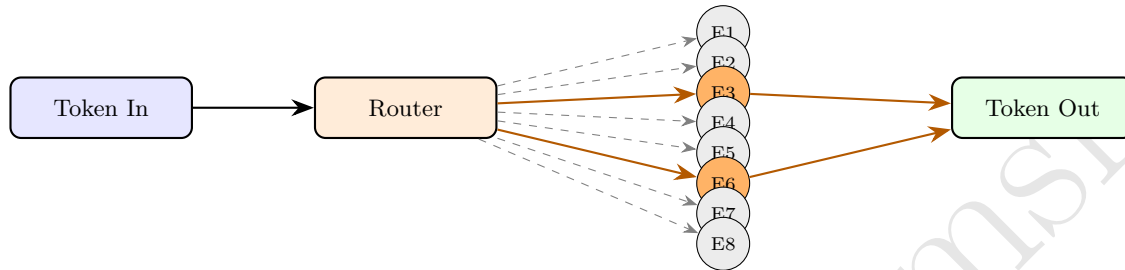


Figure 2: Inside the feedforward sub-layer only: a Mixture-of-Experts layer with 8 experts. The router (orange path) activates only 2 of the 8 experts for this token; the rest (gray, dashed) sit idle and are not even read from memory. Real models like Kimi K3 use hundreds of experts per layer, activating only a handful per token.

5.2.2 The Router, With Real Numbers

Definitions are abstract. Let us compute an actual routing decision by hand, using a tiny 4-expert example (real models use hundreds of experts, but the arithmetic is identical).

Example

Step 1: score every expert. Suppose a token’s vector (simplified to 2 dimensions so we can compute by hand) is $x = [1, 2]$. The router holds one small weight vector per expert, and computes a score for each expert as a dot product $\text{score}_i = w_i \cdot x$:

$$\begin{aligned} w_1 &= [0.5, 0.1] \Rightarrow \text{score}_1 = (0.5)(1) + (0.1)(2) = 0.7 \\ w_2 &= [0.2, 0.9] \Rightarrow \text{score}_2 = (0.2)(1) + (0.9)(2) = 2.0 \\ w_3 &= [1.0, -0.3] \Rightarrow \text{score}_3 = (1.0)(1) + (-0.3)(2) = 0.4 \\ w_4 &= [-0.1, 0.3] \Rightarrow \text{score}_4 = (-0.1)(1) + (0.3)(2) = 0.5 \end{aligned}$$

Example

Step 2: keep only the top- k . With $\text{top-}k = 2$ routing, only the two highest-scoring experts are kept: **Expert 2** (score 2.0) and **Expert 1** (score 0.7). Experts 3 and 4 are skipped entirely for this token – their weights are not loaded or computed at all, which is exactly where the compute and memory savings come from.

Example

Step 3: blend the winners. The two winning scores are turned into blending weights via softmax, so they sum to 1:

$$\text{weight}_2 = \frac{e^{2.0}}{e^{2.0} + e^{0.7}} \approx \frac{7.389}{9.403} \approx 0.786, \quad \text{weight}_1 = \frac{e^{0.7}}{e^{2.0} + e^{0.7}} \approx \frac{2.014}{9.403} \approx 0.214$$

If Expert 1 and Expert 2 each transform x into their own output (O_1 and O_2 respectively), the feedforward layer's final output is a weighted blend of just those two:

$$\text{Output} = 0.786 \times O_2 + 0.214 \times O_1$$

Expert 2 dominates the output because the router scored it far higher for this particular token. A different token – a different x – would produce different scores, and very likely route to a completely different pair of experts.

Question

Do the experts actually specialize in anything meaningful, like ‘math’ or ‘code’? Sometimes loosely, but rarely as cleanly as you would expect. Research has found some experts specialize in surface-level patterns (e.g. punctuation, specific languages) more often than clean semantic categories like ‘math.’ The router is trained purely to minimize prediction error, with no instruction to create human-interpretable specialists – any specialization that emerges is a side effect, not a design goal.

Try It Yourself

Guess the Percentage. Kimi K3 has 896 experts in each layer. For a single word, what percentage of those experts do you think actually get used? Write down your guess before reading on.

The real answer: only **16** of the 896 experts fire per token – that is $16/896 \approx 1.8\%$. If your guess was much higher than that, you are not alone; that gap between intuition and reality is exactly why sparse activation is worth understanding carefully, not just naming.

5.2.3 How Does the Router Learn Which Expert to Use?

The router is not hand-coded – it is itself a small neural network, trained end-to-end alongside the experts.

Definition

The **gating network** (router) learns to score each expert for a given token and selects the top- k highest-scoring experts. Its parameters are updated by the same backpropagation process as the rest of the model, so routing decisions improve throughout training.

Question

What stops the router from just picking the same 2 favorite experts every time? This is a real failure mode called **expert collapse**. To prevent it, MoE training adds a **load-balancing loss** – an extra penalty term that discourages the router from overusing a small subset of experts, encouraging it to spread tokens more evenly across all of them. Without this, most experts would sit permanently idle and the model would waste most of its capacity.

5.2.4 Real-World MoE Models

MoE is not just a Kimi K3 curiosity – it is now common practice among frontier labs.

Example

Named examples. **Mixtral 8x7B** (Mistral AI) uses 8 experts per layer with 2 active per token. **DeepSeek-V3** uses a much larger pool of fine-grained experts with several active simultaneously. **Kimi K3** (this lecture’s running example) uses 896 experts per layer with only 16 active. Even GPT-4 has been widely reported (though not officially confirmed by OpenAI) to use an MoE architecture. The pattern across all of them: more total capacity, without a proportional increase in the compute needed per token.

Model	Total Params	Active Params/Token	Experts (active/total)
Mixtral 8x7B	46.7B	12.9B	2 / 8
DeepSeek-V3	671B	37B	8 / 256
Kimi K3	2.8T	tens of billions (est.)	16 / 896

Table 1: Total vs. actively-used parameters per token. “Active” includes always-on components like attention (Section 5.2.1), not just the routed experts – so it is always noticeably larger than a naive expert-ratio calculation alone would suggest. Kimi K3’s exact active-parameter figure is not officially published, so it is shown here as an estimate for intuition, not a confirmed number.

Question

Why is Mixtral’s active-parameter count (12.9B) so much bigger than what a naive 2/8 ratio of its 46.7B total would suggest ($\approx 11.7B$)? The gap is exactly the always-on parts: attention layers, embeddings, and shared components are not gated by the router at all, and are included in “active” regardless of which experts are chosen. Only the feedforward experts are optional; everything else in the block runs every time.

5.3 Resolving the Kimi K3 Puzzle**Example**

Putting quantization and MoE together. Two facts, combined:

- **MoE:** Only 16 of 896 experts ($\approx 1.8\%$) activate per token. The vast majority of the model’s parameters are simply not needed for any given word, and can stay on disk rather than in RAM.
- **Quantization:** The experts that *are* needed are stored at 4 bits per weight rather than 16 or 32, quarter- or eighth-sized compared to full precision.

Combined, a model can stream only the small set of relevant, heavily quantized experts off disk for each token, rather than holding a trillion-plus full-precision parameters in memory at once. This is how Kimi K3 runs inside 8 GB of RAM – memory becomes a dial you can turn, not a hard wall.

Question

If this saves so much memory, why isn’t every model built this way? MoE introduces real engineering costs: routing decisions add complexity, load can become unbalanced across experts, and streaming experts from disk (rather than keeping everything in fast memory) can slow down generation speed – recall from the original Kimi K3 example that generation was noticeably slower at the smallest memory footprint. Efficiency and speed are a tradeoff, not a free win.

5.4 Hallucination: What and Why

Definition

Hallucination is when a model generates text that is fluent and confident, but factually incorrect or entirely fabricated.

Definition

Hallucinations are often split into two types: **intrinsic** hallucination, where the output directly **contradicts** a source the model was given (e.g. misreporting a number from a document it was just shown), and **extrinsic** hallucination, where the output adds information that is **unverifiable** against any given source at all (e.g. inventing a citation from nothing).

Hallucination is not a bug that occasionally slips through – it is a direct consequence of what an LLM fundamentally is (recall: a next-token predictor). A few common causes:

- **No built-in fact-checking.** The model predicts what token is *plausible* next, not what is *true*. A fabricated citation can look exactly as fluent as a real one.
- **Gaps in training data.** If the model was never trained on information about something obscure or recent, it will still attempt an answer rather than reliably saying “I don’t know.”
- **Overconfidence.** Models are frequently poorly calibrated – they can sound equally confident whether they are correct or completely wrong.

Try It Yourself

Spot the Hallucination. Read each statement below and decide, before looking at the answers afterward, which are true and which are hallucinated – and try to articulate *what made the false ones sound convincing*.

1. “The Transformer architecture was introduced in the paper ‘Attention Is All You Need’ in 2017.”
2. “GPT-3 has exactly 175,181,291,008 parameters.”
3. “ReAct stands for ‘Reasoning and Acting’ and interleaves Thought, Action, and Observation steps.”
4. “The Model Context Protocol was created by the ISO International Standards Committee in 2019.”
5. “Retrieval-Augmented Generation reduces hallucination by grounding responses in retrieved documents.”
6. “Studies show LLMs with more than 50 layers always outperform LLMs with fewer layers on every benchmark.”

Answers: (1) True. (2) Hallucinated – suspiciously precise; the real figure is reported simply as “175 billion,” and excessive, oddly specific precision is a common hallucination signature. (3) True. (4) Hallucinated – a fabricated authority and a date that predates the technology it describes. (5) True. (6) Hallucinated – an absolute, sweeping claim with a suspiciously round threshold and no cited source.

Across the false statements, notice the recurring pattern: fabricated precision, invented authorities or dates, and absolute claims with no source. These three red flags generalize well beyond this list.

5.5 Why Hallucination Matters More for Agents

A chatbot that hallucinates produces a wrong *sentence*. An agent that hallucinates can take a wrong *action* – and actions have consequences that a sentence alone does not.

Example

A cascading failure. Imagine a customer support agent, wired up with the tools from earlier in the course. A customer asks it to cancel “my most recent order.” The agent hallucinates an order ID instead of actually retrieving it, then confidently calls `cancel_order(id)` with the wrong ID via a tool call. The wrong customer’s order is now cancelled – a real-world, hard-to-reverse consequence, triggered by a single fabricated fact the agent never double-checked before acting.

Question

So does this mean agents are more dangerous than chatbots? Not inherently more dangerous – but definitely **higher stakes** when they fail. A chatbot’s hallucination is usually caught by a human reading the output before anything happens. An agent may act *before* any human reviews the claim at all, which is exactly why verification steps (Section 5.6) matter so much more once a system moves from generating text to taking actions.

Try It Yourself

Design a Safety Net. Return to the cancelled-order scenario above. Before reading further, try to propose **one concrete check** that could have been inserted *before* the `cancel_order` tool call executed, that would have caught the hallucinated order ID.

Three patterns worth comparing your answer against, all used in real production agent systems: confirming the order ID with the user before acting; requiring the agent to retrieve the order ID via a tool call rather than generating it from memory; or adding a human-approval step for any irreversible action.

5.6 Mitigation Strategies

Definition

Common strategies for reducing hallucination include: **grounding** responses in retrieved, verifiable documents; **prompting the model to cite sources** for its claims; **lowering sampling temperature** for factual tasks; and **verification loops**, where a second pass checks claims against a trusted source before the final answer is returned.

Definition

Self-consistency is a mitigation technique where the model is asked the same question multiple times (or via multiple reasoning paths), and the most frequently occurring answer is trusted. Hallucinated details tend to vary between attempts, while genuinely known facts tend to repeat consistently.

Question

Does this sound familiar? It should – retrieval-augmented generation, covered earlier in the course, is itself a hallucination mitigation strategy: by grounding an answer in retrieved documents rather than relying purely on memorized training data, the model has far less room to fabricate.

Example

Agent-specific mitigation. The three strategies suggested in the “Design a Safety Net” exercise above are exactly the mitigations used in real systems: requiring tool calls to retrieve facts rather than letting the model generate them from memory, confirming ambiguous or high-stakes details with the user before acting, and gating irreversible actions behind human approval.

5.7 Summary

We closed the Kimi K3 puzzle by combining two efficiency techniques: Mixture-of-Experts, which activates only a small fraction of a model's experts per token via a trained, load-balanced router, and quantization, which shrinks the memory footprint of those active experts. Together, they explain how a trillion-plus-parameter model can run in a few gigabytes of RAM – a pattern now used across several real-world models, not just Kimi K3. We then turned to hallucination – intrinsic and extrinsic, and a direct consequence of next-token prediction rather than fact-lookup – and practiced spotting it using common red flags: fabricated precision, invented authorities, and unsupported absolute claims. We argued that hallucination is higher-stakes for agents than chatbots, since a hallucinated fact can trigger a real, hard-to-reverse action, and closed with concrete mitigation strategies, including self-consistency and action-level safeguards.

Exercises

1. **MoE math.** A model has 128 experts per layer and activates 8 per token. What percentage of experts are active? If the full model has 900 billion parameters split evenly across all experts, roughly how many parameters are actively used per token?
2. **Router scoring, by hand.** A token has vector $x = [2, 1]$. Three experts have router weights $w_1 = [0.4, 0.2]$, $w_2 = [-0.1, 0.8]$, and $w_3 = [0.6, 0.6]$. Compute each expert's score, determine which expert(s) win under top- $k = 1$ routing, and explain which parts of the model besides this chosen expert still ran in full for this token.
3. **Dense vs. MoE tradeoffs.** List one advantage and one disadvantage of MoE compared to a dense architecture of the same total parameter count.
4. **Load balancing.** Explain, in your own words, why expert collapse would defeat the purpose of using MoE in the first place.
5. **Explain the puzzle.** In your own words, and without looking back at your notes, explain to a classmate how Kimi K3 runs on 8 GB of RAM. Use both quantization and MoE in your explanation.
6. **Intrinsic vs. extrinsic.** Classify each as intrinsic or extrinsic hallucination: (a) a model summarizing a document says it contains a statistic that is not actually in the document; (b) a model states a historical date that contradicts the source article it was given to summarize.
7. **Hallucination red flags.** Find (or invent) one example of a plausible-sounding but false statement an LLM might generate. Identify which red flag(s) from this lecture it exhibits.
8. **Agent cascading failure.** Describe a scenario, different from the order-cancellation example, where a hallucinated fact could cause an agent to take a harmful, hard-to-reverse action. Propose one safeguard that would have prevented it.
9. **Mitigation design.** You are building a legal research assistant where factual accuracy is critical. Propose two concrete hallucination mitigation strategies you would use, and explain why you chose them for this specific use case.